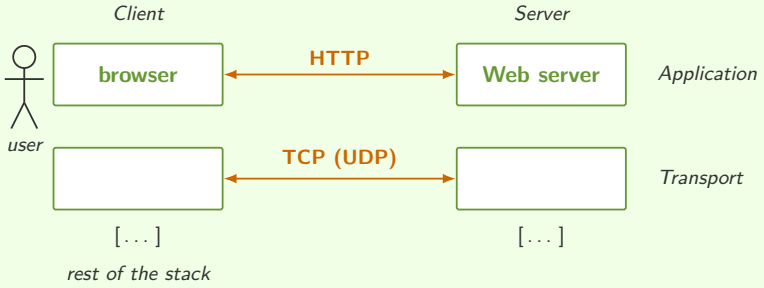
HTTP and Connection Handling

Lecture 04

Computer Networks and Distributed Programming

Where HTTP sits

Two processes at the top of the stack: the browser on the user's host, the Web server on the other. Between them, one protocol.



The horizontal arrows are logical: the data really goes down, across the network, and up again.

The application that made the Internet public

- Until the early 1990s the Internet was used mostly by researchers, academics and students: remote login, file transfer, news, e-mail.
- Those applications were useful, and still are, but outside the academic world almost nobody had heard of the network they ran on.
- The World Wide Web was the first Internet application that caught the general public's eye.
- What people liked is that it works on demand: you get what you want, when you ask for it, unlike radio and television.

It turned the Internet from one data network among many into the data network.

What the Web is, and what HTTP defines

World Wide Web

The collection of information — documents, images, video, audio — that can be accessed over the Internet according to the HyperText Transfer Protocol, HTTP.

- The exchange is client-server: a client program turns the user's requests into HTTP messages, a server program answers them.
- The client side is the browser; the server side is a program such as Apache or nginx, sitting on an always-on host.
- HTTP fixes two things only: the structure of the messages and how the two exchange them.

On top of those two things sit YouTube, webmail and most mobile applications.

A page is a set of objects

Object

A file — HTML, JPEG, JavaScript, a style sheet, a video clip — that is addressable by a single URL.

- Most Web pages are a base HTML file plus several referenced objects, and the base file names the others by their URL.
- A page with HTML text and five JPEG images therefore has six objects, not one.
- The browser cannot know the other five until it has read the first one.

So one page is several transfers — which is what the rest of this lecture is about.

The URL

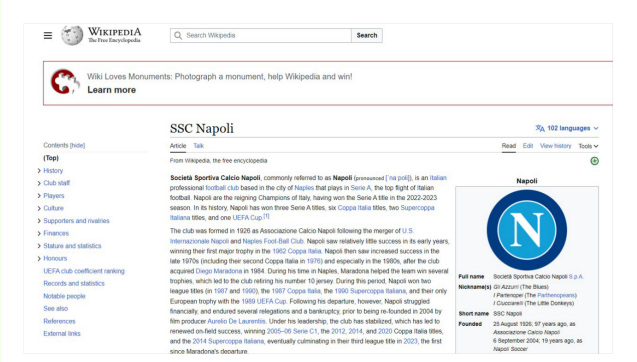
A resource is identified by a Uniform Resource Locator:

`[protocol]://[usrinfo@][host][:port][/path][?query][#fragment]`

- protocol: how the resource is to be accessed — `http`, `https`, `ftp`.
- usrinfo: username and password before an `@`; mostly deprecated, for obvious reasons.
- host: the name or the IP address of the server.
- port: usually omitted, and then inferred from the protocol: 80 for `http`, 443 for `https`.
- path: where the object sits within the server.
- query: after a `?`, what is being asked; after a `#`, an element inside the resource, which the server never sees.

A URL with a query

`https://en.wikipedia.org/w/index.php?title=SSC_Napoli`



Host `en.wikipedia.org`, path `/w/index.php`, query `?title=SSC_Napoli`: the same page as `/wiki/SSC_Napoli`, asked for in a different way.

Before the first HTTP message

What has to happen between pressing Enter and the first request going out.

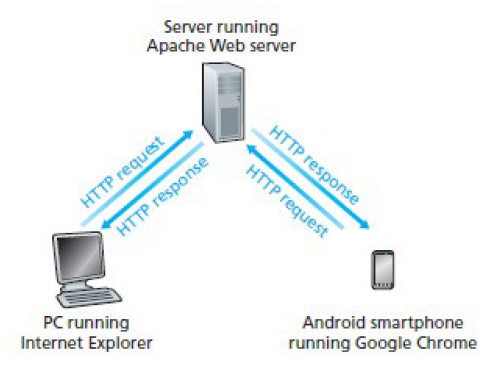
1. The browser splits the URL: protocol, host, port, path.
2. The host is a name, and no connection can be opened towards a name: the DNS supplies the address it stands for.
3. Only then a TCP connection is opened towards that address, on port 80 or 443.
4. The first HTTP request travels on that connection, carrying the path.
5. The browser reads the HTML, finds the referenced objects, and starts again.

The name lookup costs time too, and none of the round trips we count below include it.

Requests and responses

- The user asks for a page, typically by clicking a hyperlink.
- The browser sends an HTTP request message for each object in the page.
- The server answers each one with an HTTP response message carrying the object.

One server does this for many clients at once, and does not care what they are: a desktop, a phone, another program.



HTTP runs over TCP

- The client speaks first: it opens the connection towards the server, which was already listening.
- Once the connection is up, both processes reach it through their socket interfaces.
- The client writes requests into its socket and reads responses from it; the server does exactly the mirror image.
- Once a message has been written into the socket it is out of the application's hands: it belongs to TCP.

Nothing above the socket knows how many packets it took, or which route they followed.

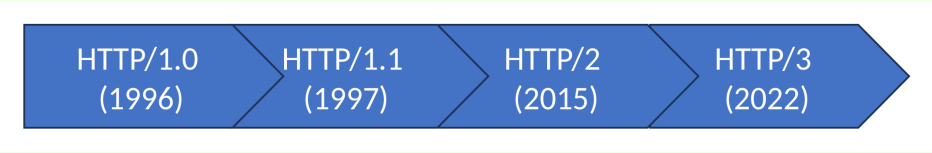
What the layer below gives, and what it costs

- TCP guarantees that each request and each response arrives intact and in order, or does not arrive at all.
- So HTTP has no sequence numbers, no acknowledgements, no retransmissions of its own: it never had to invent them.
- That is the advantage of a layered architecture, and it is why an HTTP server can be simple enough to be fast.
- The price is paid in time: a connection must be set up before anything is sent, and a missing byte holds up the bytes behind it.

Can a protocol have the reliability without paying the whole price?

Recent versions answer yes, by rebuilding reliability above UDP.

Four versions in thirty years



- HTTP/1.0: one connection per object, closed straight after the response.
- HTTP/1.1: the connection stays open and is reused; still most of the traffic.
- HTTP/2: one connection per page, with messages interleaved on it.
- HTTP/3: the same, over UDP instead of TCP.

The protocol keeps being rewritten, and every time the subject is the connection.

HTTP is stateless

Stateless protocol

The server keeps no information about its past interaction with a given client.

- Ask for the same object twice in a row and you get it twice: the server does not remember having just sent it.
- This is what lets one machine take on the order of a thousand requests per second — and a search infrastructure far more than that.
- Statelessness is a property of the protocol, not of the applications: plenty of them are stateful.

Then how does a shop remember what you put in the basket?

Persistent and non-persistent connections

Client and server usually talk for an extended period: a series of request-response pairs, back-to-back, periodic or occasional. Over a connection-oriented transport there are two ways to carry them.

The choice

- Non-persistent: a new connection for every request-response pair.
- Persistent: all requests and responses over the same connection.

HTTP/1.0 was non-persistent by default; today the default is persistent, and both ends can be configured otherwise.

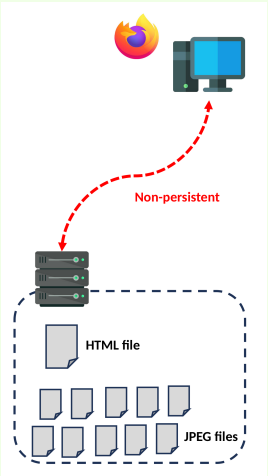
Eleven objects on one server

A page made of a base HTML file and ten JPEG images, all on the same server. The base file is at

```
http://www.someschool.edu/
someDepartment/home.index
```

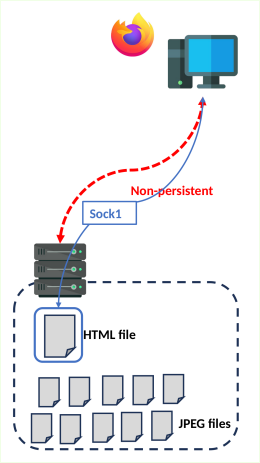
We follow it first with non-persistent connections, the way HTTP started.

How many connections does this page cost?



One object, one connection

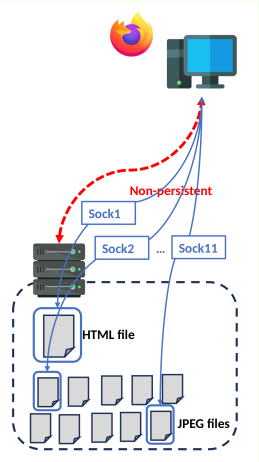
1. The client opens a TCP connection to the server on port 80: one socket at each end.
2. It sends a request carrying the path `/someDepartment/home.index`.
3. The server retrieves the object from RAM or disk, wraps it in a response message and writes it to its socket.
4. The server tells TCP to close; TCP does so only once the client has the response intact.
5. The client reads the file, finds the ten JPEG references, and starts again from step one.



Eleven connections for one page

- Eleven connections and eleven pairs of sockets, each carrying exactly one request and one response.
- The objects therefore arrive at different moments, and in an order nobody promised.

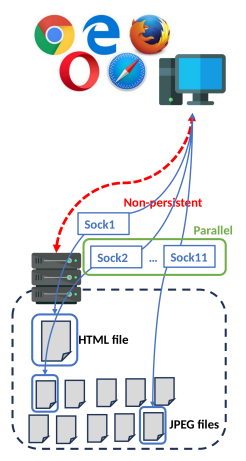
HTTP says how the two programs talk, and nothing about how the page is drawn. Two browsers may show the same page differently: one waits for everything, another paints as the pieces arrive.



In sequence or in parallel

- Once the HTML file is in, the client knows the ten images, and need not fetch them one after the other.
- Browsers typically open 5 to 10 parallel connections, each handling one request-response pair.
- Parallelism shortens the response time and costs memory and processing at both ends — the server pays for every client's choice.

Even in parallel, the setup of each connection is an overhead paid per object.



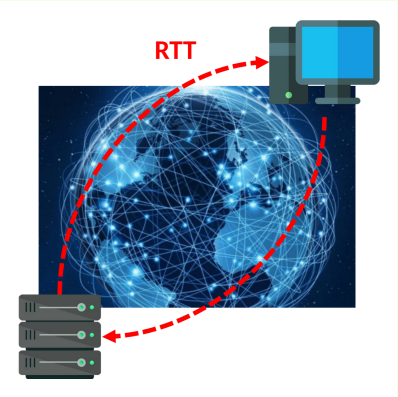
Round-trip time

RTT

The time a small packet takes to travel from client to server and back to the client.

It includes propagation on the links, queueing in the routers and switches on the way, and processing at both ends.

Timings on the Internet cannot be predicted exactly. Counting RTTs is how we compare two designs without pretending to.

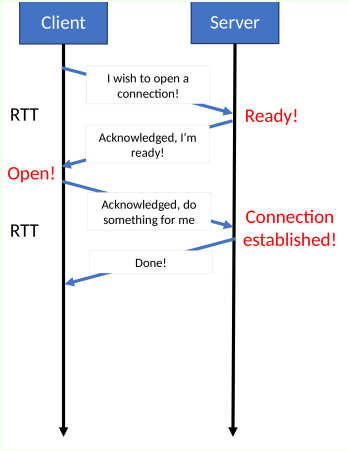


The three-way handshake

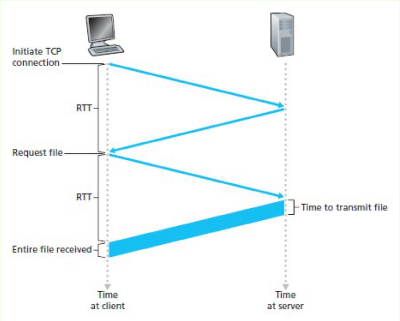
- The client sends a small segment: a connection is requested.
- The server acknowledges with a small segment of its own.
- The client acknowledges back — and in HTTP the request rides on this third segment.

So the first two parts take one RTT, and that is what opening a connection costs before any data can flow.

A similar exchange takes place when the connection is closed.



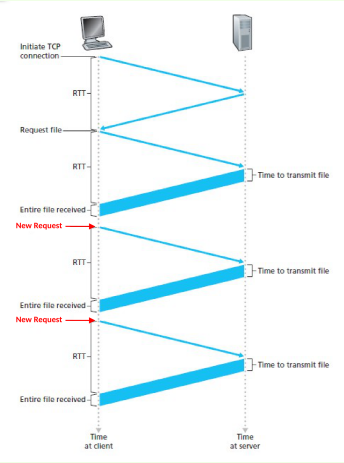
Two RTTs plus the transmission time



- One RTT for the first two parts of the handshake.
- Another RTT for the request, which travels with the third part, and for the response coming back.
- Then the time the server needs to transmit the file: that one depends on the size, not on the distance.

Persistent connections

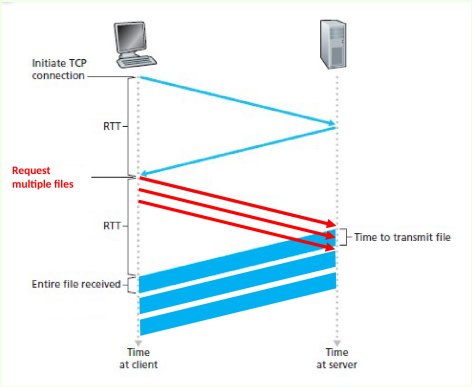
- The server leaves the connection open after sending a response.
- Further requests and responses between the same two ends travel over it: the whole page, and further pages from the same server.
- The handshake is paid once, so every object after the first costs one RTT instead of two.
- The server closes the connection when it has been unused for a configurable time.



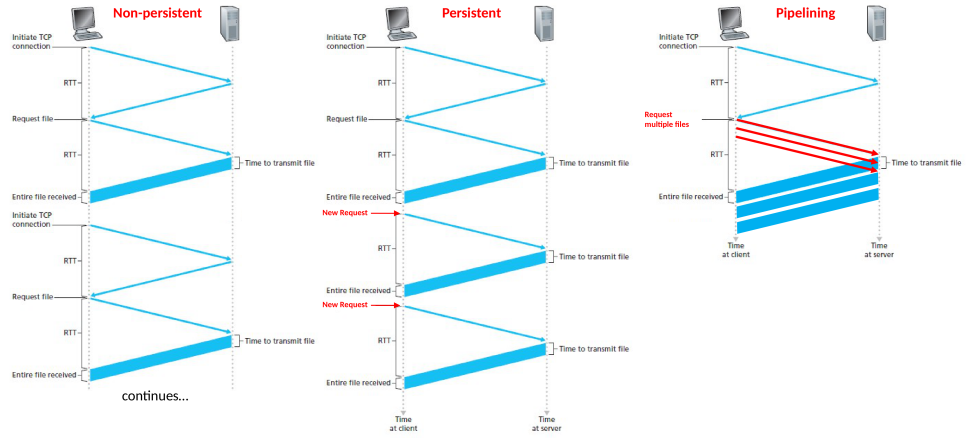
Pipelining

- Requests are issued back-to-back, without waiting for the replies to the pending ones.
- The ten images are asked for together, and the responses come back one after the other.
- Since HTTP/2 requests and responses can also be interleaved on the connection, and given priorities.

Persistent connections with pipelining are the default mode of HTTP today.



The three modes side by side



Three files, the same network. Look at where each diagram restarts: on the left at every file, in the middle never, on the right not even between the requests.

The same page, counted three ways

The eleven-object page, no losses, transmission time ignored

- Non-persistent, one at a time: $2 \times 11 = 22$ RTTs.
- Non-persistent, five in parallel: 2 for the HTML, then two rounds of 2 for the images: 6 RTTs.
- Persistent, no pipelining: 1 to open, then 1 per object: 12 RTTs.
- Persistent with pipelining: 1 to open, 1 for the HTML, 1 for the ten images: 3 RTTs.

Same page, same server, same distance. Everything between 22 and 3 is how the connections are handled.

What you pay for either choice

	Persistent	**Non-persistent**
Speed	1 RTT per object after the first	2 RTTs per object, always
Resources	one set of TCP buffers and variables	one set per object
Implementation	harder, more so with pipelining	easy, and it fits statelessness
When idle	stays open until a timeout	nothing is left open

The trade is the usual one: the connection that is already there is fast, and is a resource held for someone who may never come back.

The connection that closed while you were using it

What if the server times out just as the client reuses the connection?

- The server closes an idle persistent connection on its own, and the client is not asked.
- A request written at that same moment is lost with the connection: no response, no error from the protocol.
- So a client that reuses an idle connection must be ready to open a new one and send the request again.

Which is safe only if repeating the request is harmless — one of the reasons a `GET` is expected not to change anything on the server.

One large object at the head of the line

Why does one big object delay ten small ones?

- A page with a large video near the top, many small objects below, one persistent connection, and a slow link somewhere on the path.
- The video occupies the connection; the small objects are ready and wait behind it. This is head-of-line blocking.
- Browsers work around it by opening up to six parallel connections — which also grabs a larger share of the bottleneck, since TCP gives each connection, not each page, an equal share.

Getting rid of those parallel connections, without paying for the blocking, is what HTTP/2 is for.

HTTP/2: frames interleaved on one connection

- A framing sub-layer breaks every message into small frames: the header into one, the body into others, all binary encoded.
- Frames belonging to different responses are interleaved on the single connection, and reassembled at the other end.
- A video of 1000 frames and eight small objects of two frames each: the small ones are all delivered after 18 frames instead of 1016.
- Methods, status codes, URLs and header fields are unchanged: what changes is how the data is formatted and carried.

One connection per page again, and this time nothing waits behind the video.

Prioritization and server push

- The client can give each request a weight between 1 and 256, and declare which message it depends on.
- The server sends the frames of the highest-priority responses first: the style sheet before the image at the bottom of the page.
- With server push the server reads the base HTML, works out which objects will be needed, and sends them before they are requested.
- That removes one round trip per pushed object — the request the client was going to send anyway.

HTTP/3 and QUIC

- QUIC is a transport protocol implemented in the application layer, over the bare-bones UDP.
- It provides what HTTP needs and UDP does not: reliability, multiplexing, per-stream flow control.
- Connection setup takes fewer round trips, because the connection and its encryption are negotiated together instead of one after the other.
- A lost packet stops its own stream only: the head-of-line blocking that survived inside TCP disappears.
- HTTP/3 is HTTP over QUIC. Several HTTP/2 mechanisms are already in QUIC, so the protocol above it gets simpler.

Around 30% of HTTP traffic ran over UDP in 2024 — an order of magnitude, not a measurement.

What each version changed

Version	Connections per page	What it introduced
HTTP/1.0 (1996)	one per object	the non-persistent baseline
HTTP/1.1 (1997)	one, reused	persistent connections, pipelining
HTTP/2 (2015)	one, frames interleaved	binary framing, priorities, push
HTTP/3 (2022)	one, over QUIC on UDP	setup in fewer round trips

Thirty years of work on a single question: how many connections, and for how long.

Summary

- A Web page is a set of objects, each with its own URL, and each fetched separately.
- HTTP defines the messages and the rules of exchange, and nothing about how a page is displayed.
- It runs over TCP and inherits reliability, paying for it in round trips.
- It is stateless: nothing of one exchange survives into the next.
- Non-persistent means a connection per object, two RTTs each; persistent pays the handshake once, and one RTT per object after.
- Pipelining removes the waiting between requests; HTTP/2 interleaves them, HTTP/3 moves the whole thing onto QUIC.

Next we open the messages themselves, and see what those two lines actually contain.

Installing Wireshark

On Debian, Ubuntu and derivatives. Answer yes when asked whether non-root users may capture packets.

```
$ sudo apt update && sudo apt install wireshark
$ sudo dpkg-reconfigure wireshark-common    if you answered no
$ sudo usermod -aG wireshark $USER          then log out and back in
$ groups                                    wireshark must appear
$ getcap /usr/bin/dumpcap
cap_net_raw,cap_net_admin=eip               what it must print
```

If `getcap` prints nothing, assign those two capabilities to the binary yourself, with `setcap`.

Capturing your first GET

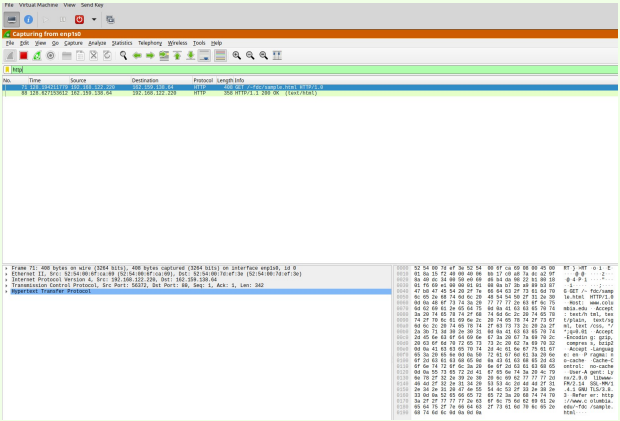
1. Start Wireshark as set up on the previous slide, without starting the capture.
2. Type `http` in the display filter, so that only HTTP packets are shown.
3. Start capturing.
4. Fetch a very simple page — the first web page, still served as it was:

```
$ lynx http://info.cern.ch/hypertext/WWW/TheProject.html
```

5. Stop the capture.

Collapse the Frame, Ethernet, IP and TCP details, and expand the HTTP one.

What you should see



Two HTTP packets: the GET your client sent and the 200 OK that came back — sitting inside a TCP segment, an IP datagram and an Ethernet frame.

Questions: the request

(a) Which HTTP version does your client announce? And which one does the server answer with?

(b) Which languages does the client tell the server it is willing to accept?

(c) What is the IP address of your machine, and what is the server's?

The answers are all in the two packets you have just captured — read them off the capture, not off these slides.

Questions: the response

(d) What status code did the server return?

(e) When was the page last modified on the server?

(f) How many bytes of content were sent to your client?

(g) In the raw bytes of the packet, are there header lines that the packet list does not show?

On (e): the date is 3 December 1992. Nothing on this page has changed since, which is exactly what makes it worth caching — the subject of the next lecture.

Then repeat with `http://scottaaronson.com`: the answer is a 301, and the browser goes on to HTTPS. And repeat once more with a modern browser instead of `lynx`: what changed in the request, and why?